

Phase 1 — Feedback Management System

1. Project Title

Feedback Management System

2. Project Overview

Organizations, training institutes, educational platforms, and enterprises regularly collect feedback from participants, employees, customers, or trainees to evaluate the quality of services, training programs, products, and overall experience.

Currently, many organizations collect feedback using:

- Google Forms
- Excel sheets
- Emails
- Manual surveys

These approaches often result in:

- Scattered feedback data
- Difficult reporting and tracking
- No centralized management
- Limited search capability
- No structured storage for future analytics

To improve the feedback collection and management process, the organization wants to build a centralized web-based **Feedback Management System** using:

- **Frontend:** React
- **Backend:** Python FastAPI
- **Database:** SQLite/PostgreSQL

This Phase 1 implementation focuses on developing the foundational full-stack application with core feedback management capabilities. The system should also be designed to support future extensions involving:

- Data Engineering pipelines
- Feedback analytics dashboards
- Sentiment analysis
- AI-powered semantic search
- GenAI-based summarization systems

3. Business Problem Statement

The organization currently lacks a centralized mechanism for managing and analyzing feedback data.

Current challenges include:

- Feedback stored across multiple platforms
- Difficulty identifying common issues
- Manual consolidation of responses
- No efficient search capability
- Limited historical analysis
- Poor visibility into participant satisfaction trends

The proposed application will centralize feedback collection, storage, search, and management into a single platform.

4. Project Objective

The objective of this capstone project is to design and develop a scalable Feedback Management System that allows users to:

- Submit feedback
- View submitted feedback
- Search feedback records
- Filter feedback based on ratings or categories
- Manage feedback data centrally

The application should follow industry-standard full-stack development practices and REST API design principles.

5. Scope of Phase 1

Phase 1 includes:

- Full-stack application development
- REST API implementation
- Feedback CRUD operations
- Search and filtering functionality
- Database integration
- Frontend-backend integration

Phase 1 does NOT include:

- Authentication and authorization
- AI/ML integration
- Sentiment analysis

- Dashboard analytics
- Semantic search
- Cloud deployment

6. Functional Requirements

6.1 Submit Feedback

The system should allow users to submit feedback by providing:

- Participant Name
- Training/Event/Product Name
- Rating
- Feedback Comments

6.2 View Feedback

The system should allow users to:

- View all feedback records
- View detailed feedback entries
- View ratings and comments
- View submission timestamps

6.3 Update Feedback

Administrators should be able to:

- Modify feedback details
- Update ratings
- Edit comments if necessary

6.4 Delete Feedback

The system should allow deletion of feedback entries when required.

6.5 Search and Filter Feedback

The application should support:

- Keyword-based search
- Rating-based filtering
- Training/Event-based filtering

7. Rating Levels

The system should support ratings between:

- 1 to 5

Example:

- 1 → Poor
- 2 → Fair
- 3 → Good
- 4 → Very Good
- 5 → Excellent

8. System Users

User Type	Responsibilities
Participant/User	Submit feedback
Administrator	Manage feedback records

9. Technical Requirements

9.1 Frontend Requirements (React)

The frontend application should be developed using React.

Mandatory Concepts

- Functional Components
- React Hooks
- Component-based architecture
- Axios or Fetch API
- Form validation
- Responsive UI design

Suggested Pages

Dashboard Page

Displays:

- Total feedback count
- Average rating
- Recent feedback entries

Submit Feedback Page

Allows users to submit feedback.

Feedback Listing Page

Displays all feedback entries.

Feedback Details Page

Displays complete feedback information.

Search & Filter Section

Allows filtering based on:

- Rating
- Training/Event/Product
- Keywords

9.2 Backend Requirements (FastAPI)

The backend application should be developed using FastAPI.

Mandatory Concepts

- FastAPI routing
- Pydantic models
- CRUD operations
- Database integration
- Exception handling
- REST API design
- CORS middleware

10. API Requirements

Method	Endpoint	Description
GET	/feedback	Retrieve all feedback
GET	/feedback/{id}	Retrieve feedback by ID
POST	/feedback	Submit feedback
PUT	/feedback/{id}	Update feedback
DELETE	/feedback/{id}	Delete feedback
GET	/search	Search feedback

11. Database Requirements

The application should use:

- SQLite (preferred for Phase 1)
OR
- PostgreSQL

12. Database Schema

Feedback Table

Column Name	Data Type
feedback_id	Integer
participant_name	String
program_name	String
rating	Integer
comments	Text
submitted_at	DateTime

13. Suggested Project Structure

Backend Structure

```
backend/  
|  
|— main.py  
|— database.py  
|— models.py  
|— schemas.py  
|— crud.py  
|— routers/  
|   |— feedback.py  
|— services/  
|— requirements.txt
```

Frontend Structure

```
frontend/  
|  
|— src/  
|   |— components/  
|   |— pages/  
|   |— services/  
|   |— App.js  
|   |— api.js
```

14. Non-Functional Requirements

The application should:

- Be responsive across devices
- Handle invalid inputs gracefully
- Follow modular coding practices
- Maintain clean UI design

- Support future scalability

15. Expected Deliverables

Participants are expected to submit:

Source Code

- Complete React frontend
- Complete FastAPI backend

Database

- Database schema/scripts

Documentation

- README file
- Installation/setup guide
- API documentation

Screenshots

- UI screenshots
- API testing screenshots

16. Evaluation Criteria

Criteria	Weightage
Frontend Development	20%
Backend API Development	25%
Database Integration	15%
CRUD Functionality	15%
Search & Filtering	10%
Code Quality & Structure	10%
Documentation	5%

17. Recommended Technology Stack

Layer	Technology
Frontend	React
Backend	FastAPI
Database	SQLite/PostgreSQL

Layer	Technology
-------	------------

API Testing	Postman
-------------	---------

Version Control	Git/GitHub
-----------------	------------

18. Recommended Development Timeline

Phase	Duration
-------	----------

Requirement Analysis	0.5 Day
----------------------	---------

Backend Development	1 Day
---------------------	-------

Frontend Development	1 Day
----------------------	-------

Integration & Testing	0.5 Day
-----------------------	---------

Documentation	0.5 Day
---------------	---------